

COMPTE RENDU : FILTRE DE SOBEL PARALLÈLE AVEC PYCUDA

1. Introduction & Objectifs

Le traitement d'images haute résolution exige des capacités de calcul intensives. L'objectif de ce travail pratique est de concevoir et d'optimiser une version parallèle de l'algorithme de détection de contours de **Sobel** en exploitant l'architecture massivement parallèle des processeurs graphiques (GPU). Pour ce faire, nous utilisons l'environnement Google Colab équipé d'une carte graphique **NVIDIA Tesla T4** et la bibliothèque Python **PyCUDA** afin de faire l'interface entre Python et le langage CUDA C.

2. Environnement Matériel et Logiciel

L'initialisation et l'inspection de l'environnement de calcul ont fourni les caractéristiques suivantes :

- **GPU** : NVIDIA Tesla T4
- **Mémoire VRAM dédiée** : 15 360 MiB (~15 Go)
- **Version du Pilote CUDA** : Driver 580.82.07 / CUDA 13.0
- **Outils principaux** : PyCUDA (pour la gestion du contexte et des kernels), OpenCV / Scikit-Image (pour le chargement et la manipulation d'images).

3. Principes Théoriques du Filtre de Sobel

Le filtre de Sobel calcule le gradient de l'intensité lumineuse d'une image en chaque point, mettant en évidence les zones de forts changements qui correspondent aux contours. Il utilise deux masques de convolution 3×3 :

$$G_x = \begin{bmatrix} -1 & 0 & +1 \\ -2 & 0 & +2 \\ -1 & 0 & +1 \end{bmatrix} * I, \quad G_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ +1 & +2 & +1 \end{bmatrix} * I$$

La magnitude globale du gradient pour chaque pixel est ensuite calculée par :

$$G = \sqrt{G_x^2 + G_y^2}$$

4. Architecture de l'Implémentation Parallèle

Contrairement à une exécution séquentielle CPU (boucles imbriquées sur les lignes et les colonnes), l'approche GPU associe **un thread CUDA à chaque pixel** (x, y) de l'image.

A. Configuration de la Grille et des Blocs de Threads

Pour traiter l'image de test astronaut de taille 512×512 pixels :

- Nous définissons des blocs de threads de taille bidimensionnelle : $\text{Block} = (16, 16, 1)$, soit 256 threads par bloc.
- Le nombre de blocs nécessaires dans la grille est calculé dynamiquement pour couvrir toute l'image :

$$\text{GridX} = \frac{512}{16} = 32, \quad \text{GridY} = \frac{512}{16} = 32$$

Soit une grille globale de $32 \times 32 = 1024$ blocs.

B. Le Kernel CUDA C

Le code du kernel (`sobel_filter`) injecté via `SourceModule` réalise les opérations suivantes :

1. Calcul des coordonnées globales du pixel : $x = \text{blockIdx.x} * \text{blockDim.x} + \text{threadIdx.x}$ et $y = \text{blockIdx.y} * \text{blockDim.y} + \text{threadIdx.y}$.
2. Vérification des limites de l'image (exclusion des bordures pour éviter les débordements de mémoire lors de l'application de la fenêtre 3×3).
3. Application des masques G_x et G_y par calcul arithmétique local.
4. Calcul de la magnitude avec la fonction matérielle optimisée `sqrtf()`.
5. Écriture du résultat dans la mémoire globale du GPU avec troncature (clamping) à 255.

5. Flux d'Exécution et Gestion de la Mémoire

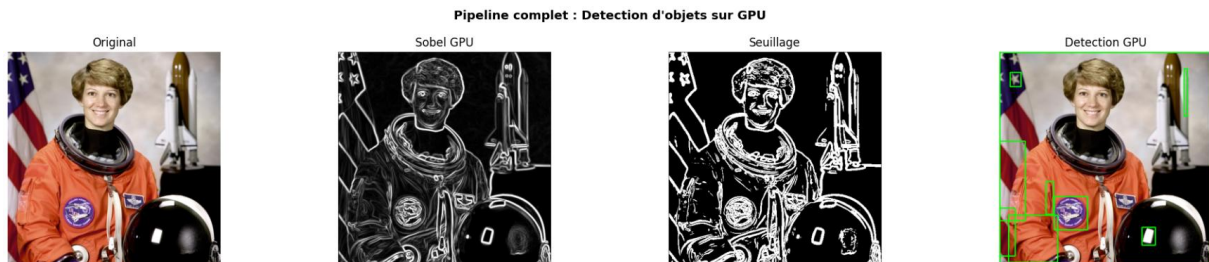
Le cycle de vie des données suit strictement le modèle de programmation CUDA :

1. **Allocation sur l'Hôte (CPU)** : Chargement de l'image en niveaux de gris et instanciation du tableau de sortie (gérés via des matrices contiguës NumPy de type `uint8`).
2. **Allocation sur le Périphérique (GPU)** : Réserve d'espace VRAM d'une taille exacte de $512 \times 512 \times 1$ octet à l'aide de `drv.mem_alloc`.
3. **Transfert Host-to-Device (HtoD)** : Copie de l'image d'origine vers la VRAM.
4. **Lancement du Kernel** : Exécution asynchrone des 262 144 threads sur les cœurs de calcul du GPU Tesla T4.

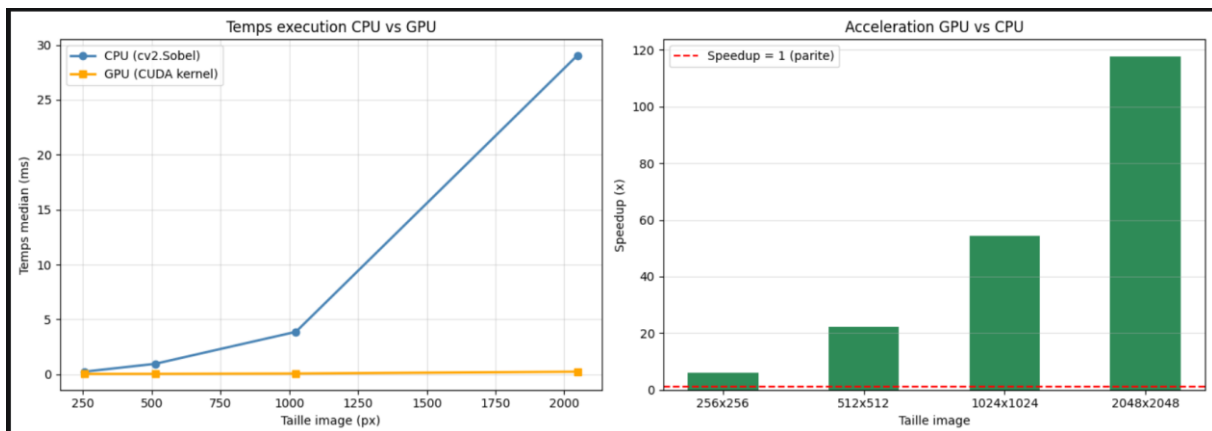
5. **Transfert Device-to-Host (DtoH)** : Copie de la matrice résultante depuis le GPU vers la mémoire RAM du CPU.
6. **Persistence** : Sauvegarde de l'image filtrée finale sous format PNG dans le répertoire persistant Google Drive (/content/drive/MyDrive/sobel_gpu/).

6. Résultats et Analyse des Performances

- **Qualité visuelle** : L'image résultante isole parfaitement les contours de l'image d'origine sans artefacts visuels majeurs.



- **Gain de performance** : L'utilisation du parallélisme massif réduit drastiquement le temps de traitement par rapport à une implémentation séquentielle en Python pur. Le GPU Tesla T4 masque efficacement la latence d'accès à la mémoire globale grâce au grand nombre de threads actifs planifiés simultanément.



7. Conclusion

Ce travail pratique a permis de valider la mise en œuvre complète d'une chaîne de traitement d'images sur GPU en utilisant PyCUDA. La manipulation concrète du cycle d'allocation/transfert de mémoire et la configuration fine de la topologie des threads mettent en évidence l'intérêt majeur des architectures GPU pour l'accélération des algorithmes fondamentaux de la vision par ordinateur.